



U.S. General Services
Administration



Agentic AI Book Club - Session 3

Reviewing Chapter 1.4 & 1.5

US General Service Administration
AI Community of Practice - March 2026

Agenda

- Housekeeping
 - Chapter review
 - Quiz review
 - Community sharing: Why did you embark on this journey?
 - Intro to next week's chapters
- 

Housekeeping

- **Agentic AI is an early and rapidly evolving knowledge domain**
 - Study plan overview (strategic repeats were built-in)
 - Supplemental repository overview
 - All Chapter texts are available
 - Chapter 1.3 quiz review - Don't be discouraged by hard questions (and here's why)
- 

Chapter 1.3 Quiz review

Quiz Review

Which protocol is specifically designed for workflow orchestration in multi-agent systems: ACP (Agent Communication Protocol), A2A (Agent-to-Agent), or MCP (Model Context Protocol)?

Note:

- <https://lfai.data.foundation/communityblog/2025/08/29/acp-joins-forces-with-a2a-under-the-linux-foundations-lf-ai-data/>
 - <https://dotsquarelab.com/resources/acp-and-a2a-united>
- 
- A decorative graphic in the bottom right corner consisting of several overlapping blue rectangular shapes of varying shades, creating a sense of depth and movement.

Quiz Review

Choose between centralized and decentralized orchestration for a 15-agent data processing workflow. The workflow is well-defined and sequential, but individual agents occasionally fail. Which pattern is appropriate and why?

Hint:

- Revisit section “Orchestration and Control Patterns”: centralized orchestration and decentralized orchestration
- Given that agents occasionally fail, which pattern makes error handling and recovery simpler?

Note:

- Tricky answer choices may begin with a correct detail to be followed by incorrect detail(s) - Please read and understand the entire answer choices.
- 

Quiz Review

Design communication architecture for a 20-agent distributed analytics system with two patterns: (1) 5 agents need synchronous request-response for workflow coordination requiring latency $<100\text{ms}$ and strong consistency, and (2) 15 agents need many-to-many status notifications where eventual consistency is acceptable and delivery can be delayed up to 1 second. Evaluate message passing, blackboard, event-driven, and API-based mechanisms using selection criteria.

Hint:

- Which mechanism gives you direct, point-to-point request-response with explicit contracts and strong consistency, and can realistically meet a sub-100ms round-trip when designed correctly?
 - For many-to-many notifications where occasional delay is fine, which mechanism is explicitly described as decoupling publishers and subscribers, supporting fan-out, and tolerating eventual consistency?
- 
- A decorative graphic in the bottom right corner consisting of several overlapping blue rectangles of varying shades, creating a sense of depth and movement.

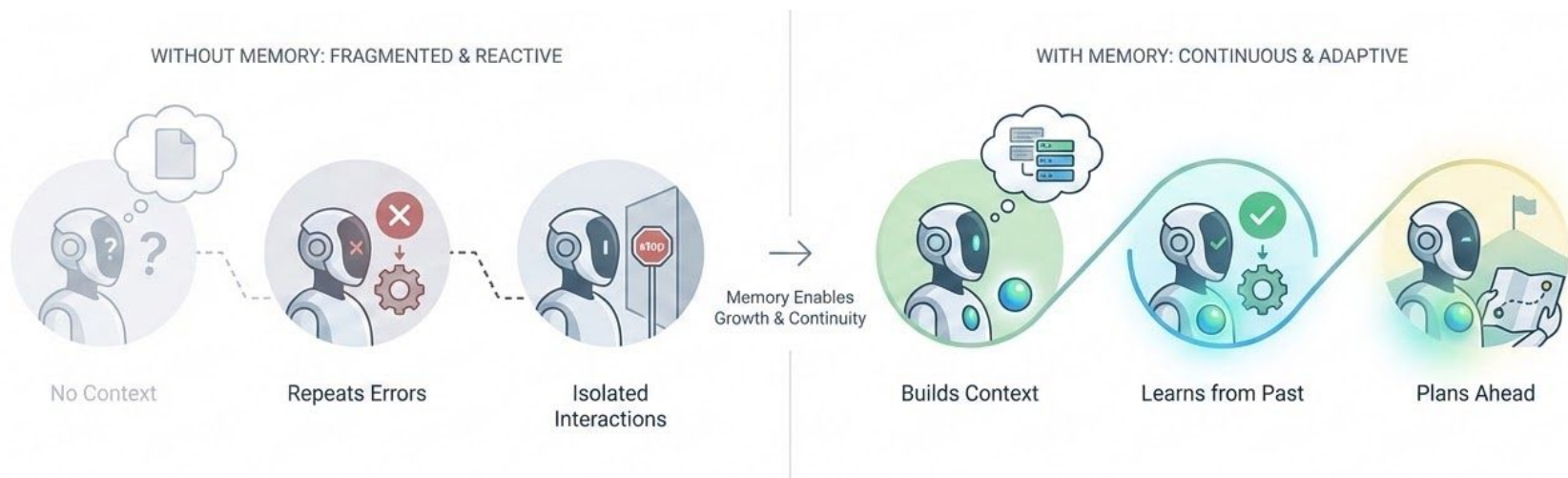
Quiz Review

A system has 8 microservices labeled as 'agents' that deploy together as a unit, share a single PostgreSQL database, and require synchronized version updates. Is this an appropriate multi-agent architecture?

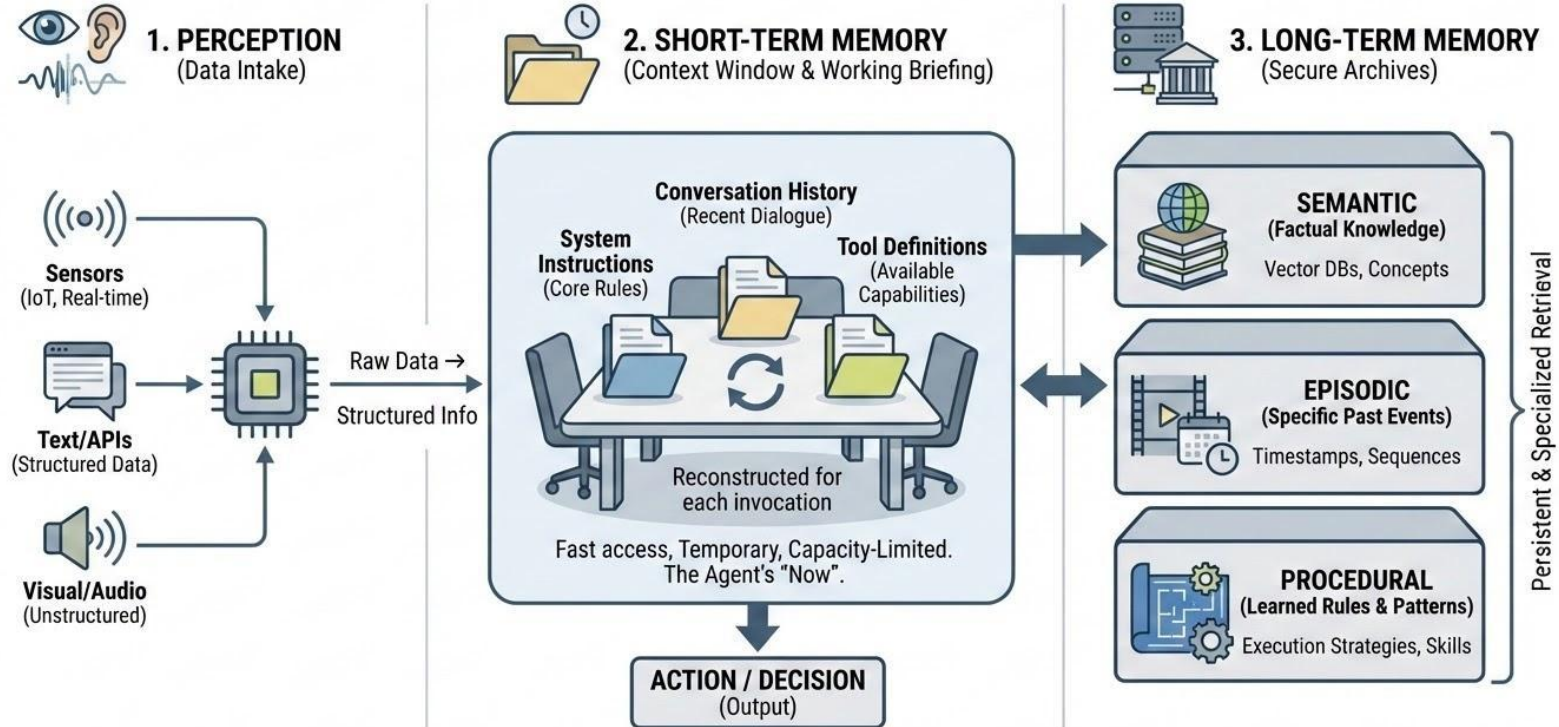
Hint:

- Check the anti-pattern descriptions
 - Does what described fit any of the anti-patter or not?
- 

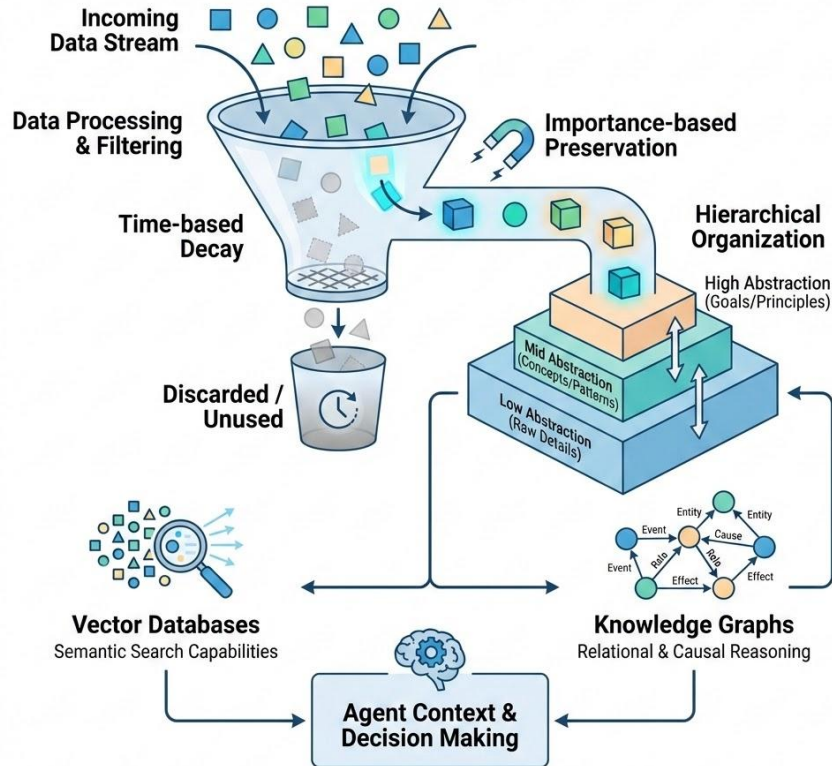
Why Memory?



Agents' Memory System

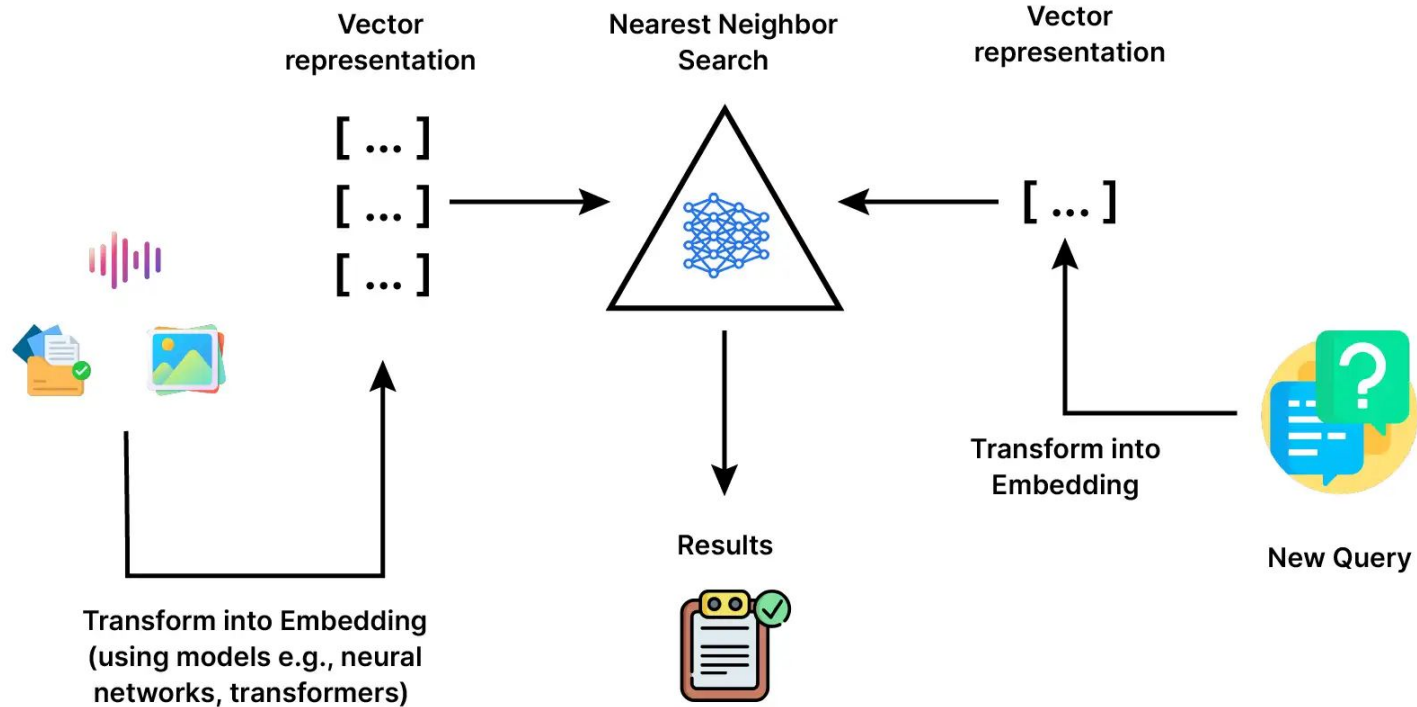


Memory Lifecycle Management



- Time-based decay removes unused data
- Importance-based preservation retains valuable context
- Hierarchical organization across abstraction levels
- Vector databases enable semantic search capabilities
- Knowledge graphs add relational and causal reasoning

Vector Store Misconceptions



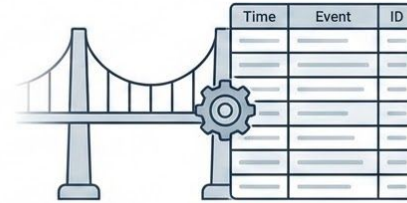
Vector Store Misconceptions (cont.)



EXCEL AT:
Semantic Similarity Search

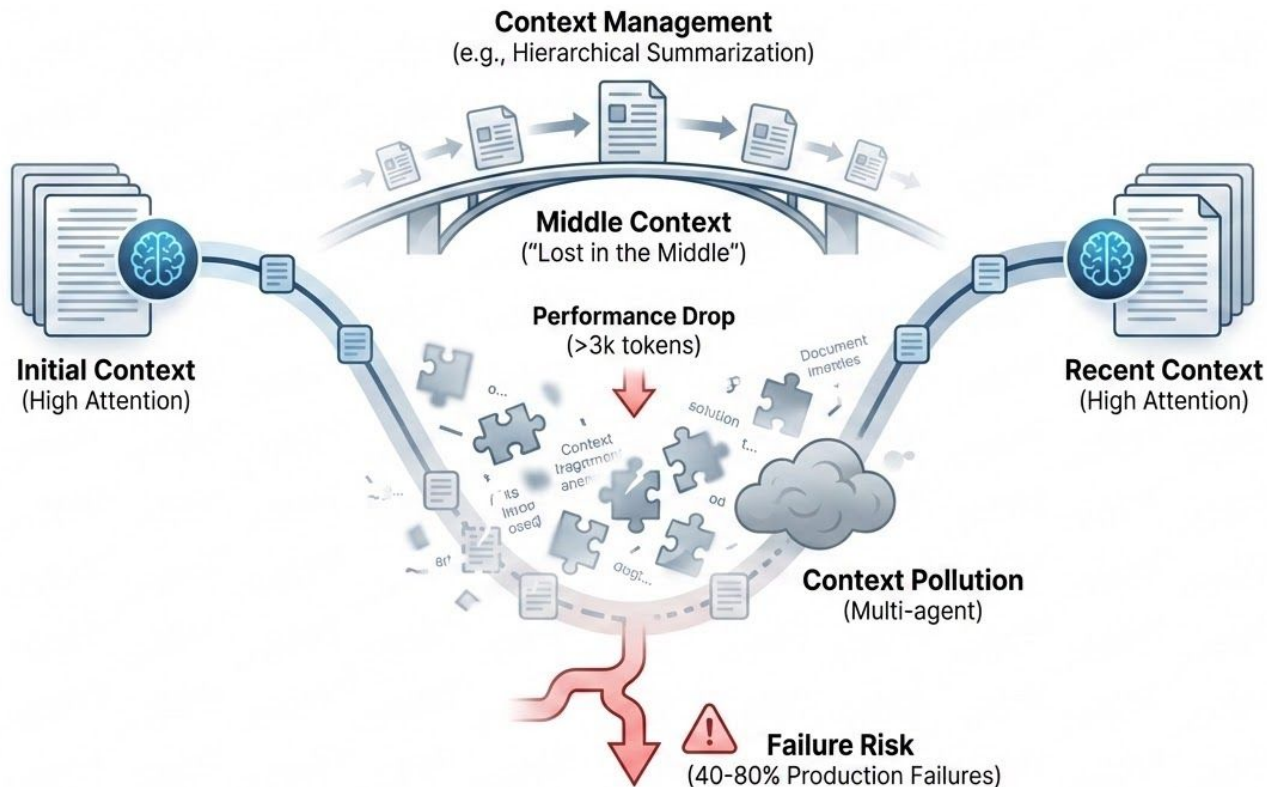


FAIL AT:
Sequences & Chronology



SOLUTION:
Hybrid Architectures
(Vector + Relational Stores)

The "Lost in the Middle" issue



Quiz Review

A developer notices their agent's performance degrades around 3,000 tokens despite having an 8,000-token context window. The agent receives full conversation history plus all retrieved memories. What hierarchical context management strategy should they implement?

A:

- {{quiz question}}

Note:

- {{notes relating to the quiz question}}
- 

What could go wrong if we don't manage multiple agents memory and perceptions well?

Why Stateful Orchestration?

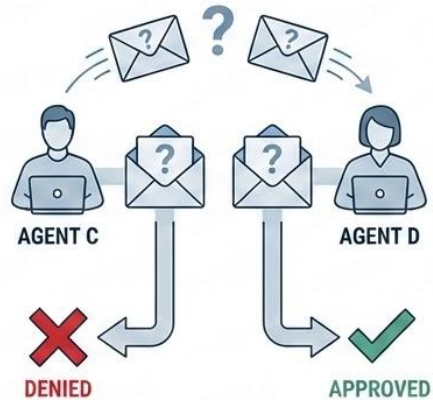
LOSS OF CONTEXT / MEMORY



Agent A completes Step 1 but has no way to inform Agent B. Agent B fails Step 2 due to missing context.

OUTCOME: INCOMPLETE TASK, REDUNDANT WORK

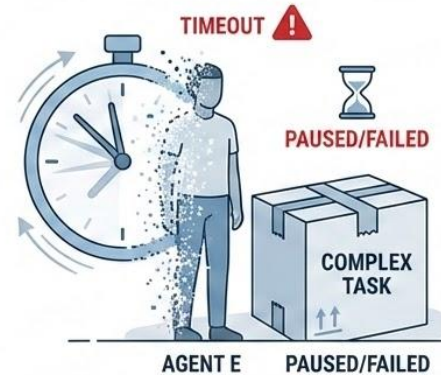
INCONSISTENT DECISION MAKING



Without shared history, different agents process identical requests, leading to conflicting results.

OUTCOME: CONFLICTING RESULTS, ERRORS

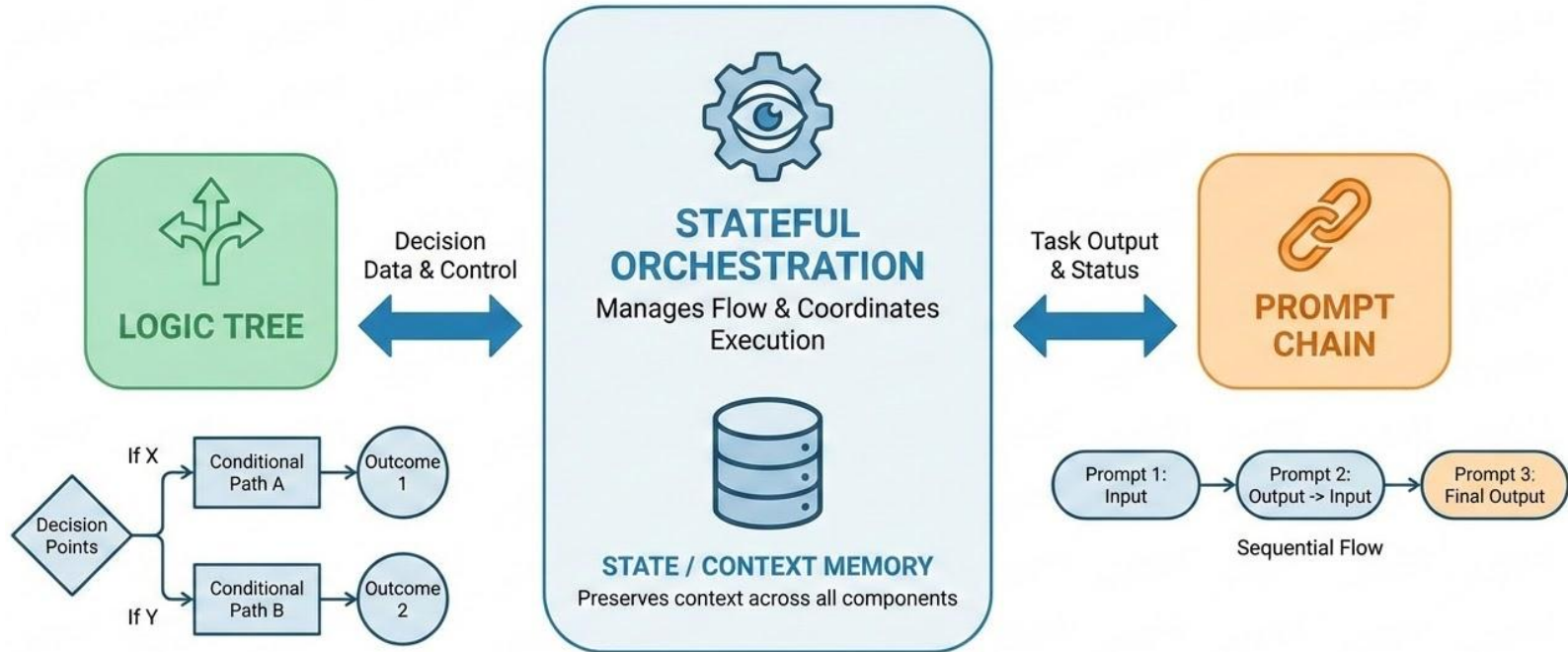
INABILITY TO HANDLE LONG-RUNNING PROCESSES



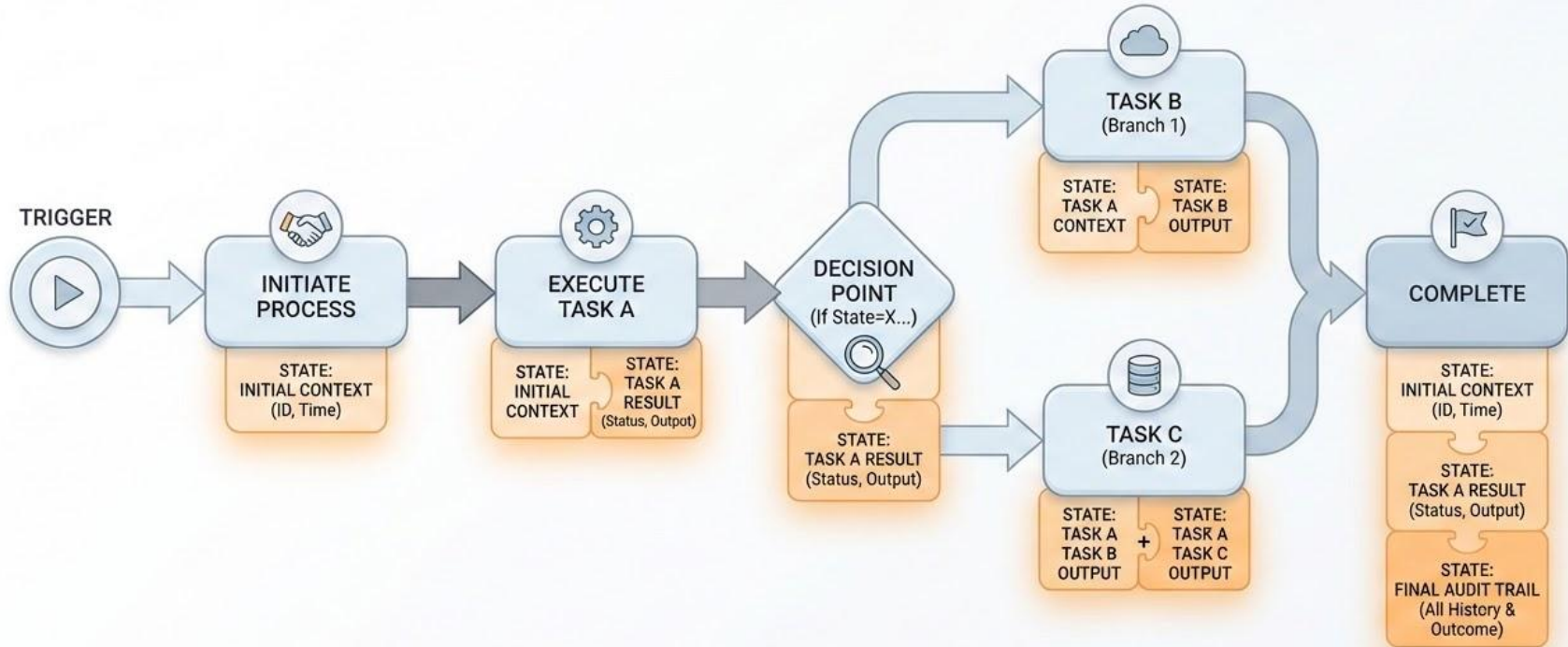
Stateless agents cannot "pause and resume" complex, long-running tasks, leading to timeouts.

OUTCOME: PROCESS FAILURE, TIMEOUTS

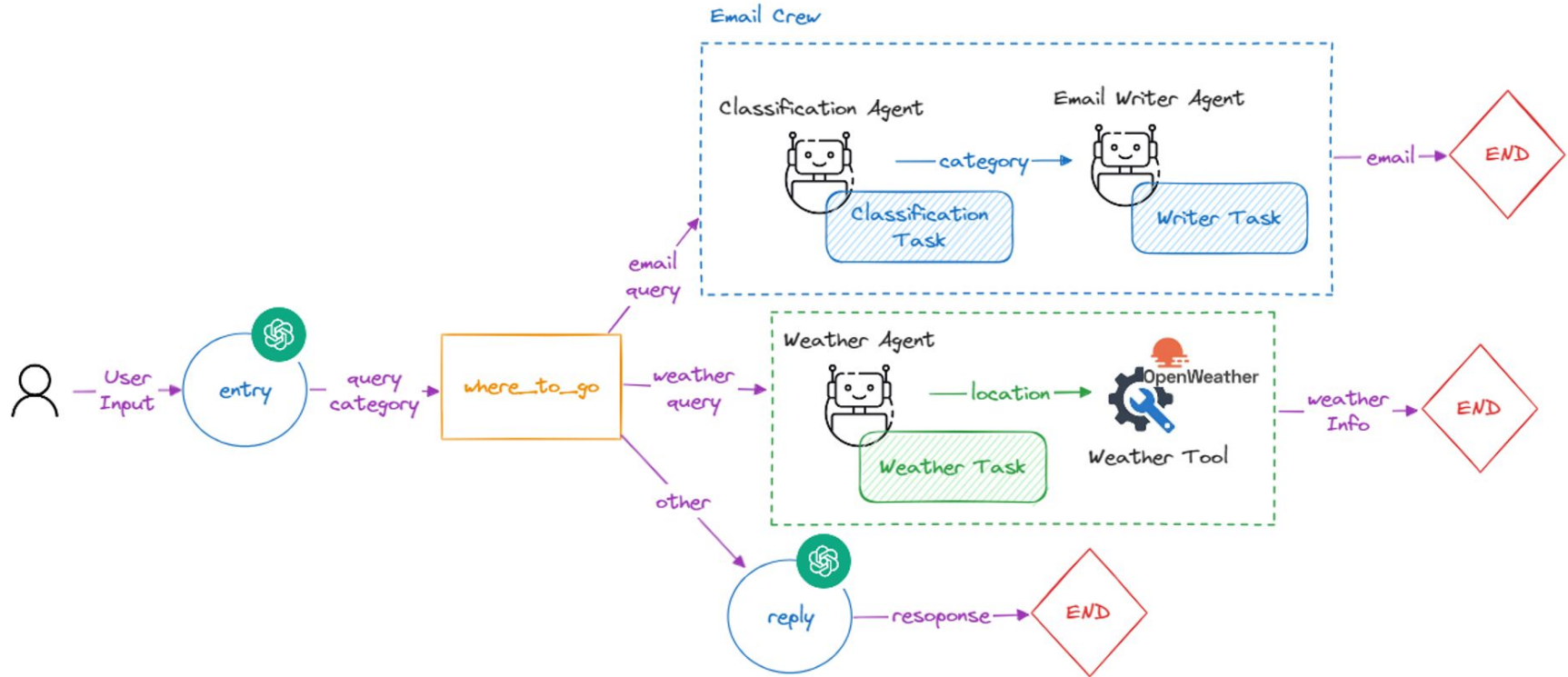
Logic Tree & Prompt Chain



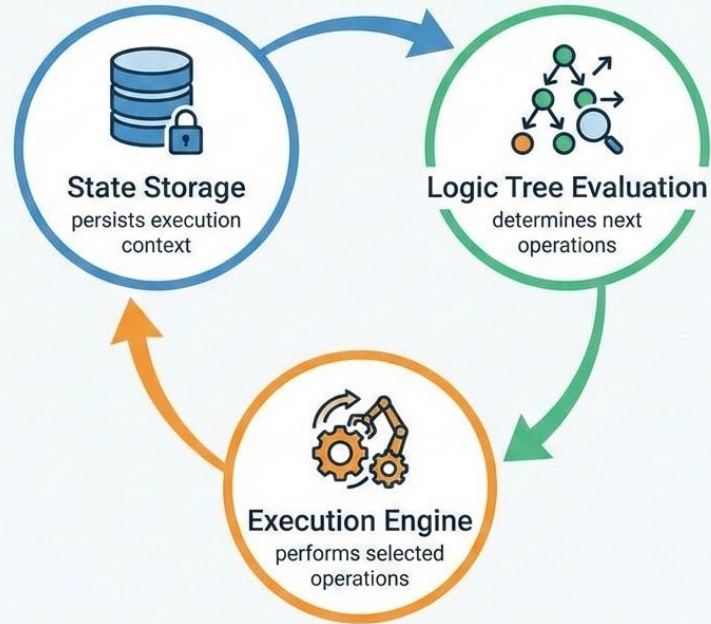
Stateful Orchestration



State Machines



The Stateful Orchestration Framework



Supported by memory managers and tool integration

Principles for Stateful Orchestration

1 Explicit transitions

Traceable audit trails. Failed workflows resume from last successful transition

2 Idempotent operations

Produce same result if repeated. Enables safe retry logic in distributed systems

3 Separated state data and control flow decisions

Data and context live in one place. Decision logic and workflow structure live in another

Worked Example

A production workflow is failing intermittently, but logs only show 'State changed from A to B'. Debugging is difficult because you cannot determine why transitions occurred. What architectural element is missing?

Why this journey?
How is your morale?

Next Chapters

- Chapter 1.6: Stateful Orchestration - Pitfalls, Integration, and Synthesis

Drill down further on stateful orchestration, ties all previous chapters together.

- Chapter 1.7A: Relational Reasoning with Knowledge Graphs

Introduces knowledge graphs as structured representations of entities, relationships, and properties, addressing limitations of vector-based retrieval systems where relationships are implicit in embeddings. Demonstrates how knowledge graphs complement vector RAG for questions requiring explicit relationship traversal and multi-hop reasoning.